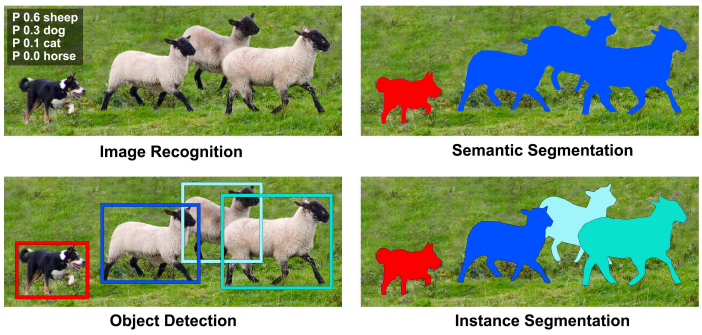
TEL328 - Procesamiento Digital de Imágenes

Clase 11: Segmentación semántica y de instancias

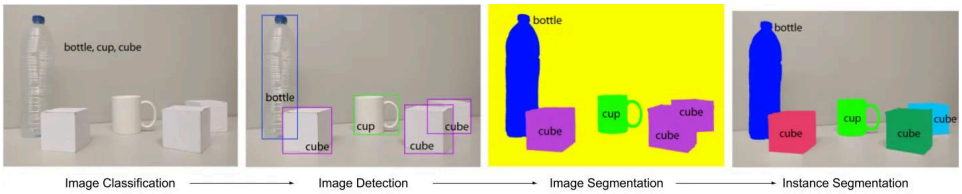
Prof. Nicolás Torres
nicolas.torresr@usm.cl
Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

Introducción

La segmentación semántica es la técnica de separar la imagen a nivel de píxeles, pero con un nivel de "comprensión" de los objetos. En otras palabras, involucra el análisis y la clasificación de cada píxel en múltiples clases (etiquetas).



La segmentación de instancias presenta un nivel de granularidad mayor que la semántica porque distingue los diferentes objetos de una misma clase. Por lo tanto, el modelo sabe cuántas instancias hay presentes de cada clase. Puede pensar en esto como aplicar un detector de objetos encima de una capa de segmentación semántica.



In [1]:
```
import numpy as np
import skimage
import skimage.io
import skimage.transform
import matplotlib.pyplot as plt

from keras.applications.vgg16 import VGG16
from keras.applications.vgg16 import preprocess_input
from keras.applications.vgg16 import decode_predictions
```

Clasificación de imágenes

In [2]:
```
vgg16_model = VGG16(weights='imagenet', include_top=True)
```

In [3]:
```
def recognition(model, filename, height=224, width=224, K=10):
    image = skimage.io.imread(filename)
    image_resize = skimage.transform.resize(image,(height,width),preserve_ran
    x = np.expand_dims(image_resize, axis=0)
    x = preprocess_input(x)
    predictions = model.predict(x, verbose = 0)
    top_K = decode_predictions(predictions, top=K)[0]
    plt.figure(figsize = (10,10))
    plt.axis('off')
    plt.imshow(image)
    plt.show()
    for _, name, y_proba in top_K:
            print("{:.2f}%\t{:12s}".format(y_proba * 100, name))
```

In [4]:
```
recognition(vgg16_model, "test/catdog.jpg")
```



```
19.20%  German_shepherd
9.59%   Norwegian_elkhound
6.38%   Egyptian_cat
6.03%   tabby
2.52%   kelpie
2.29%   tiger_cat
1.92%   tennis_ball
0.87%   totem_pole
0.87%   timber_wolf
0.84%   Eskimo_dog
```

In [5]:
```
import matplotlib.colors
```

In [6]:

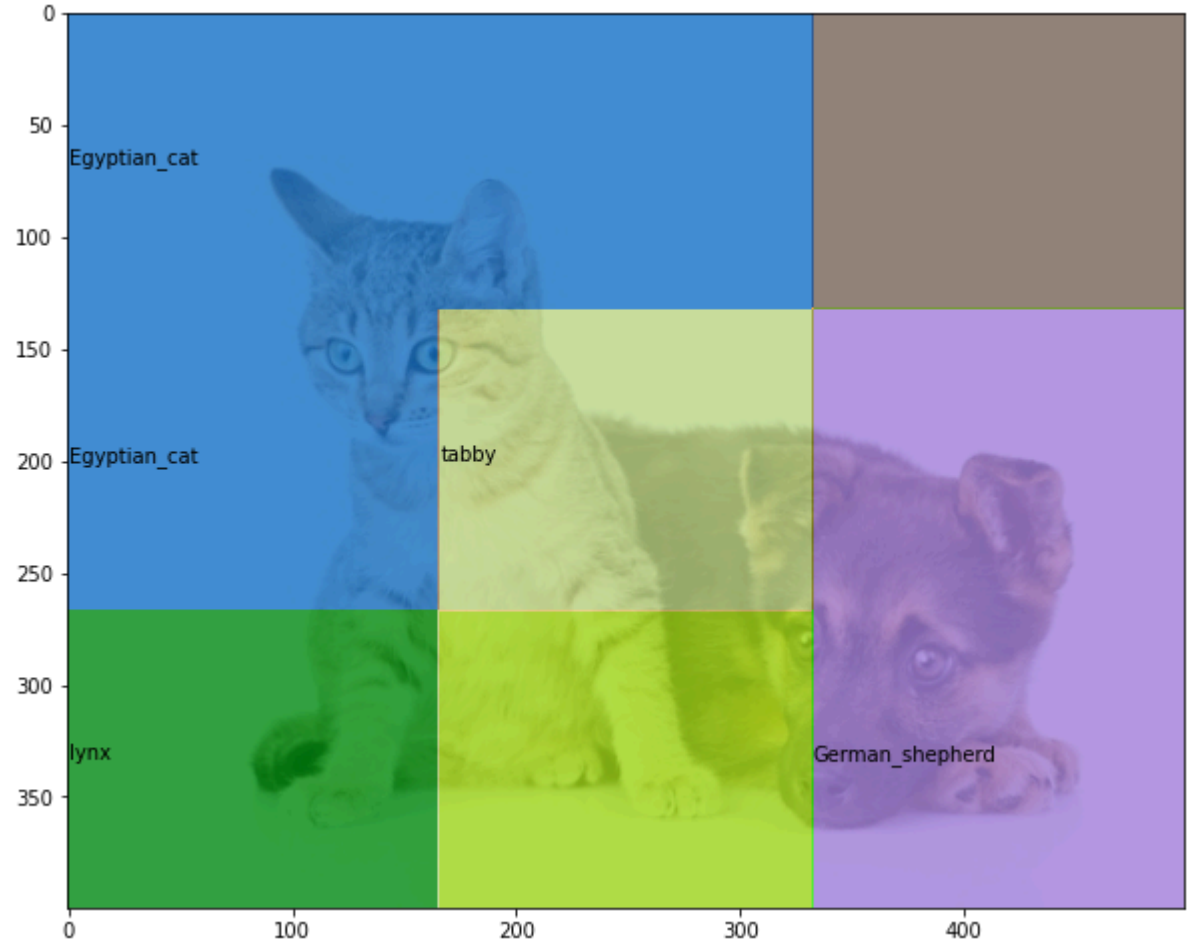
```
def cnn_segmentation(model,filename,rows=2,cols=2,patch=400,step=100,height=2
    image = skimage.io.imread(filename)
    all_features = []
    M = image.shape[0]//rows
    N = image.shape[1]//cols
    for i in range(0,M*rows-1,M):
        for j in range(0,N*cols-1,N):
            subimage = image[i:i+M,j:j+N,:]
            image_resize = skimage.transform.resize(subimage,(height,width),p
            x = np.expand_dims(image_resize, axis=0)
            x = preprocess_input(x)
            features = model.predict(x, verbose = 0)
            all_features.append(features)
    filtered = [decode_predictions(x, top=K)[0][0][1] if decode_predictions(x
                for x in all_features ]
    cmap = matplotlib.colors.ListedColormap ( np.random.rand ( 256,3))
    reshaped = np.reshape([np.argmax(x) for x in all_features], (rows,cols))
    names = np.reshape(filtered, (rows,cols))
    plt.figure(figsize=(10,10))
    plt.imshow(image)
    plt.imshow(skimage.transform.resize(reshaped,(image.shape[0],image.shape[
                                        order=0,preserve_range=True),cmap = c
    for x in range(names.shape[0]):
        for y in range(names.shape[1]):
            plt.text(x=(y)*image.shape[1]/reshaped.shape[1],y=(x+0.5)*image.s
    plt.show()
```

Segmentación de características

Usando una CNN tradicional y separando el espacio de características.

In [7]:

```
cnn_segmentation(vgg16_model, "test/catdog.jpg", rows=3,cols=3)
```



Diferencias con detección de objetos

La principal dificultad en esta tarea es que cuando las imágenes pasan por una CNN normal, pierden gradualmente su resolución espacial, por lo que una CNN normal puede terminar sabiendo que hay una persona en la imagen, en algún lugar en la parte inferior izquierda, pero no será mucho más precisa que eso.

Para realizar la segmentación semántica, se requiere un mayor nivel de comprensión de la imagen. El algoritmo debe averiguar los objetos presentes y también los píxeles que corresponden al objeto.

Segmentación semántica

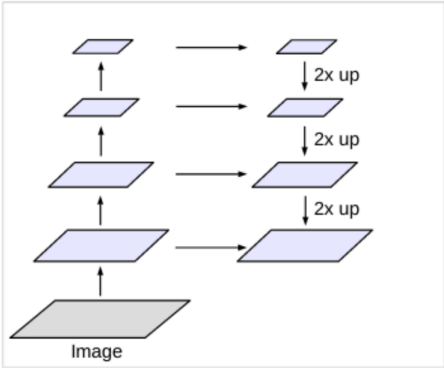
Arquitectura general

La arquitectura de la segmentación semántica puede entenderse en términos generales como una red codificadora seguida de una red decodificadora:

- El codificador suele ser una red de clasificación preentrenada como VGG/ResNet seguida de una red decodificadora.
- La tarea del decodificador es proyectar semánticamente las características discriminatorias (resolución más baja) aprendidas por el codificador en el espacio de píxeles (resolución más alta) para obtener una clasificación densa.

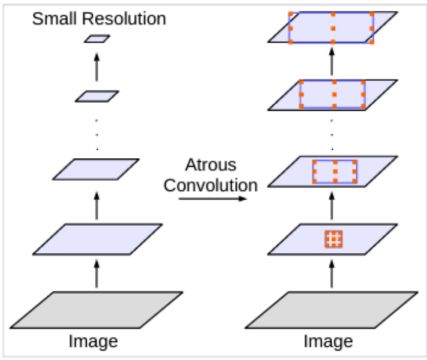
Encoder-Decoder

El codificador es responsable de obtener los mapas de características de las imágenes de entrada y el decodificador ayuda a recuperar gradualmente los detalles del objeto y las dimensiones espaciales.



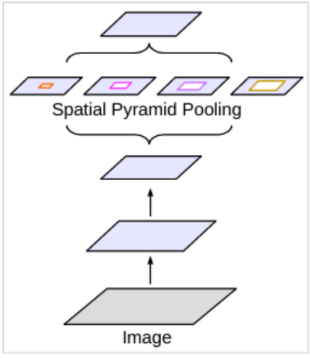
Atrous Convolution

Las CNN usan convoluciones y max-pooling que dan como resultado mapas espaciales de muy baja resolución. Las convoluciones atrosas (o dilatadas) generan kernels con ceros entremedio para aumentar el tamaño de los mapas y poder capturar un área más grande.



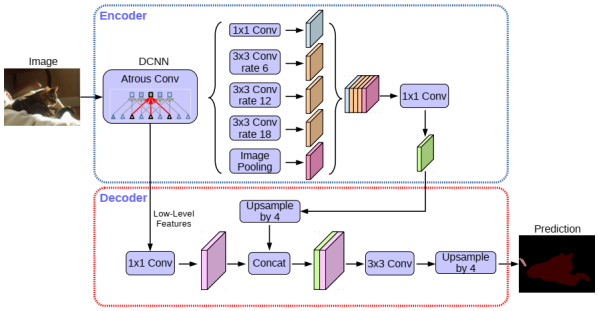
Spatial Pyramid Pooling (SPP)

Spatial Pyramid Pooling usa capas de pooling a diferentes escalas para lidiar con imágenes y objetos de tamaños variables.



DeepLab

DeepLab es un modelo de segmentación semántica de última generación de código abierto diseñado por Google.



In [2]:
```
import torch
from PIL import Image
from torchvision import transforms
from torchvision.models.detection import maskrcnn_resnet50_fpn, MaskRCNN_ResN
from torchvision.utils import draw_bounding_boxes
from torchvision.utils import draw_segmentation_masks
import torchvision.transforms.functional as F
from torchvision.io import read_image
```

In [ ]:
```
deeplabv3_model = torch.hub.load('pytorch/vision:v0.10.0', 'deeplabv3_resnet5
deeplabv3_model = deeplabv3_model.eval()

# Otras variantes disponibles:
# model = torch.hub.load('pytorch/vision:v0.10.0', 'deeplabv3_resnet101', pre
# model = torch.hub.load('pytorch/vision:v0.10.0', 'deeplabv3_mobilenet_v3_la
```

El modelo PyTorch DeepLabV3 ResNet50 ha sido entrenado en el conjunto de datos MS COCO. Pero en lugar de entrenar con todas las 80 clases, el modelo de segmentación se entrenó con solo las 20 clases correspondientes al conjunto de datos PASCAL VOC. Las siguientes son las categorías:

```
['__background__', 'aeroplane', 'bicycle', 'bird', 'boat',
'bottle', 'bus',  'car', 'cat', 'chair', 'cow', 'diningtable',
'dog', 'horse', 'motorbike',  'person', 'pottedplant', 'sheep',
'sofa', 'train', 'tvmonitor']
```

In [4]:
```
n_classes = 21
palette = np.random.uniform(0, 255, size=(n_classes, 3))
colors = torch.as_tensor([i for i in range(n_classes)])[:, None] * palette
colors = (colors % 255).numpy().astype("uint8")
```

In [5]:
```
def deeplab_segmentation(model, filename, colors):
    input_image = Image.open(filename)
    input_image = input_image.convert("RGB")
    preprocess = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0

    input_tensor = preprocess(input_image)
    input_batch = input_tensor.unsqueeze(0)

    if torch.cuda.is_available():
        input_batch = input_batch.to('cuda')
        model.to('cuda')
```

```
    with torch.no_grad():
        output = model(input_batch)['out'][0]
    output_predictions = output.argmax(0)

    r = Image.fromarray(output_predictions.byte().cpu().numpy()).resize(input
    r.putpalette(colors)

    fig = plt.figure(figsize=(15, 10))
    ax1 = fig.add_subplot(121)
    plt.imshow(input_image)
    plt.axis('off')

    ax2 = fig.add_subplot(122)
    plt.imshow(r)
    plt.axis('off')
    plt.show()
```

In [19]:

```
deeplab_segmentation(deeplabv3_model, "test/catdog.jpg", colors)
```



In [14]:

```
import os
import re
```

In [14]:

```
def segmentar_todo(model, directory, colors):
    for filename in os.listdir(directory):
        f = os.path.join(directory, filename)
        if re.search("\.(jpg|png)$", filename):
            input_image = Image.open(f)
            input_image = input_image.convert("RGB")
            preprocess = transforms.Compose([
            transforms.ToTensor(),
            transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.22
            ])

            input_tensor = preprocess(input_image)
            input_batch = input_tensor.unsqueeze(0)

            if torch.cuda.is_available():
                input_batch = input_batch.to('cuda')
                model.to('cuda')

            with torch.no_grad():
                output = model(input_batch)['out'][0]
            output_predictions = output.argmax(0)
```

```
            r = Image.fromarray(output_predictions.byte().cpu().numpy()).resi
            r.putpalette(colors)

            fig = plt.figure(figsize=(15, 10))
            ax1 = fig.add_subplot(121)
            plt.imshow(input_image)
            plt.axis('off')

            ax2 = fig.add_subplot(122)
            plt.imshow(r)
            plt.axis('off')
            plt.show()
```

In [15]:

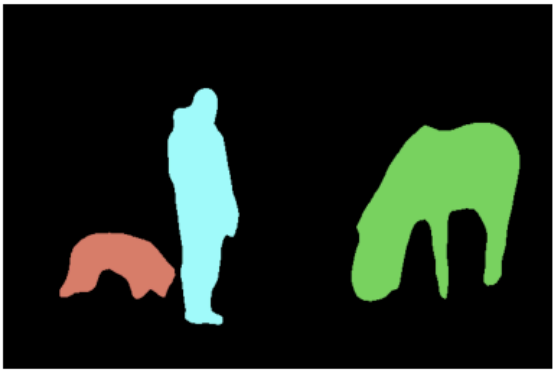
```
segmentar_todo(deeplabv3_model, "test", colors)
```



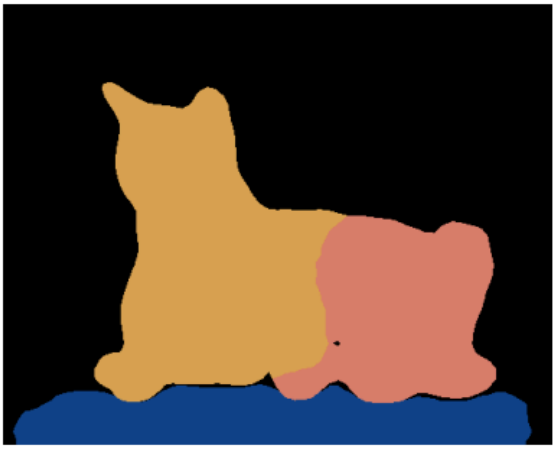


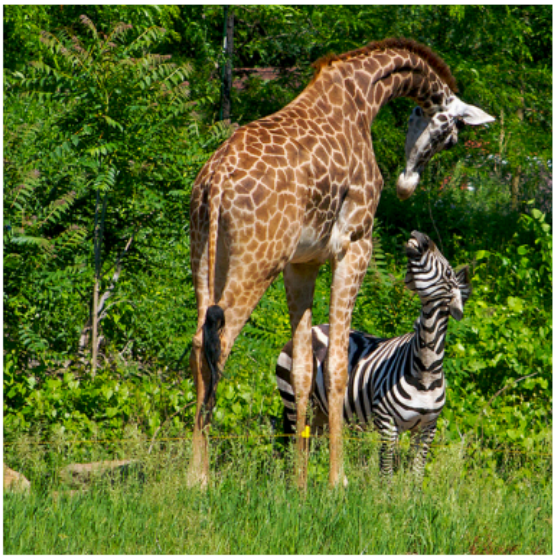


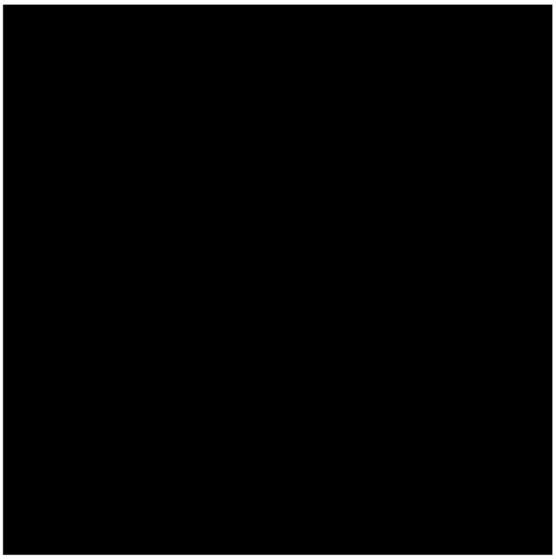
















Segmentación de instancias

Mask R-CNN

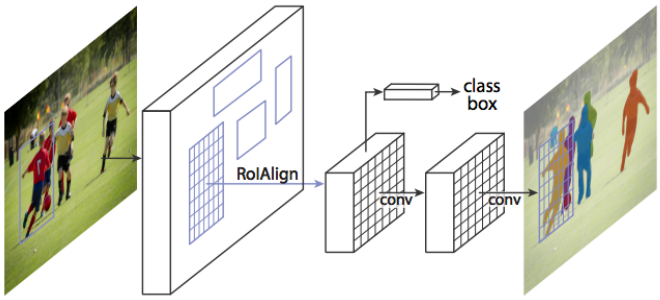
> Mask R-CNN detecta objetos individuales y, para cada objeto detectado, genera una máscara de píxeles.

Instance Segmentation = Detection + Mask

Segmentación de instancias

Mask R-CNN

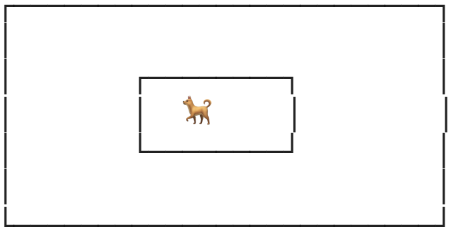
Extiende la arquitectura R-CNN al agregar una capa que genera máscaras de objetos.



RoI: Region of Interest

Es una región propuesta.

Por ejemplo:



El problema es que las regiones pueden tener diferentes tamaños:

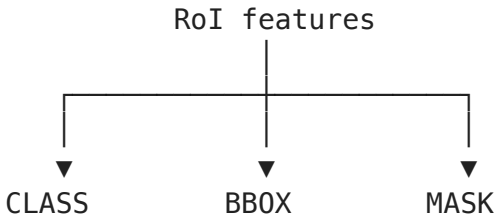
- RoI 1 = 37 × 52
- RoI 2 = 81 × 94
- RoI 3 = 25 × 31

> RoI Align transforma cada región en algo como: 7 × 7 × channels

Después de RoI Align tenemos algo como:

RoI: 7 × 7 × channels

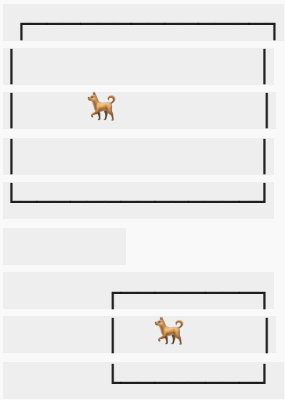
Y desde aquí salen tres ramas:



¿Qué clase es?

background	0.01
person	0.02
dog	0.93
car	0.01
cat	0.03

¿Dónde está exactamente?



¿Qué píxeles pertenecen al objeto?

`RoI → Mask Head → máscara`

Mask Head: La máscara no se produce simplemente mediante una clasificación normal.

`RoI features → Conv → Conv → Conv → Conv → Upsampling → mask`

In [9]:
```
weights = MaskRCNN_ResNet50_FPN_Weights.DEFAULT
maskrcnn_model = maskrcnn_resnet50_fpn(weights=weights, progress=False)
maskrcnn_model = maskrcnn_model.eval()
```

In [10]:
```
coco_names = ['__background__', 'person', 'bicycle', 'car', 'motorcycle', 'ai
    'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'N/A', 'stop s
    'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep', 'cow',
    'elephant', 'bear', 'zebra', 'giraffe', 'N/A', 'backpack', 'umbrella', 'N
    'handbag', 'tie', 'suitcase', 'frisbee', 'skis', 'snowboard', 'sports bal
    'kite', 'baseball bat', 'baseball glove', 'skateboard', 'surfboard', 'ten
    'bottle', 'N/A', 'wine glass', 'cup', 'fork', 'knife', 'spoon', 'bowl',
    'banana', 'apple', 'sandwich', 'orange', 'broccoli', 'carrot', 'hot dog',
    'donut', 'cake', 'chair', 'couch', 'potted plant', 'bed', 'N/A', 'dining
    'N/A', 'N/A', 'toilet', 'N/A', 'tv', 'laptop', 'mouse', 'remote', 'keyboa
    'microwave', 'oven', 'toaster', 'sink', 'refrigerator', 'N/A', 'book',
```

```
    'clock', 'vase', 'scissors', 'teddy bear', 'hair drier', 'toothbrush'
]
```

In [11]:

```
def process_image(filename,height=500,width=500):
    image = read_image(filename)
    return [transforms.Resize(size=(height,width), antialias=True)(image)]
```

In [12]:

```
def filter_model_output(output,score_threshold = .8):
    filtred_output = list()
    for image in output:
        filtred_image = dict()
        for key in image.keys():
            filtred_image[key] = image[key][image['scores'] >= score_threshol
        filtred_output.append(filtred_image)
    return filtred_output
```

In [13]:

```
def get_boolean_mask(output):
    for index,pred in enumerate(output):
        output[index]['masks'] = pred['masks'] > 0.5
        output[index]['masks'] = output[index]['masks'].squeeze(1)
    return output
```

In [14]:

```
def mask_segmentation(model,filename,class_names,score_threshold=0.8,height=5
    image_list = process_image(filename,height,width)
    transforms = weights.transforms()
    images = [transforms(d) for d in image_list]
    output = model(images)
    output = filter_model_output(output=output,score_threshold=score_threshol
    output = get_boolean_mask(output)

    for index, (image, prediction) in enumerate(zip(image_list, output)):
        draw_mask = draw_segmentation_masks(image, prediction.get('masks'), a

        fig = plt.figure(figsize=(15, 10))
        z = fig.add_subplot(121)
        img = F.to_pil_image(image)
        plt.imshow(np.asarray(img))
        plt.axis('off')

        a = fig.add_subplot(122)
        width = image.shape[2]
        height = image.shape[1]
        img = F.to_pil_image(draw_mask)
        a.imshow(np.asarray(img))
        a.axis('off')
        a.set(xticklabels=[], yticklabels=[], xticks=[], yticks=[])
        boxes = prediction.get('boxes')
        scores = torch.round(prediction.get('scores'), decimals=1)
        for i in range(len(boxes)):
            box = boxes[i]
            x1 = int(box[0]+(box[2]-box[0])//2)
            y1 = int(box[3]+(box[1]-box[3])//2)
            conf = scores.detach().numpy()[i]
            name = class_names[prediction.get('labels')[i]] + "(" + str(conf)
            a.text(x1 , y1, name, fontsize = 16, color='white',
                   bbox=dict(alpha=0.5, facecolor='black', edgecolor='black')
    plt.show()
```

In [20]:
```
mask_segmentation(maskrcnn_model, "test/horses.jpg", coco_names)
```



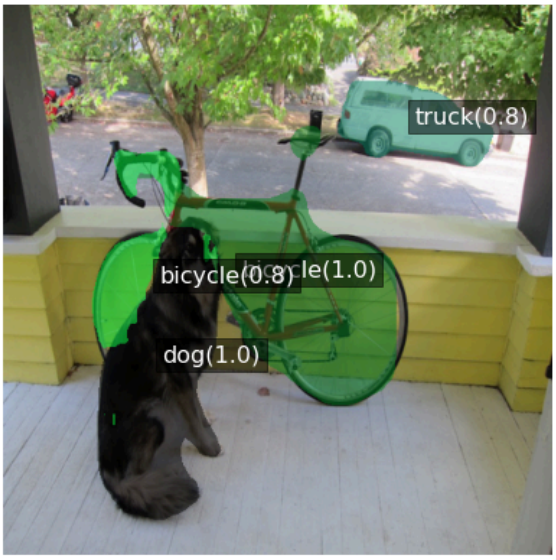
In [11]:
```
mask_segmentation(maskrcnn_model, "test/person.jpg", coco_names)
```



In [12]:
```
def mask_segmentation_all(model,directory,class_names,score_threshold=0.8,hei
    for filename in os.listdir(directory):
        f = os.path.join(directory, filename)
        if re.search("\.(jpg)$", filename):
            mask_segmentation(model, f, class_names, score_threshold = score_
```

In [15]:
```
mask_segmentation_all(maskrcnn_model, "test", coco_names, score_threshold=.75
```

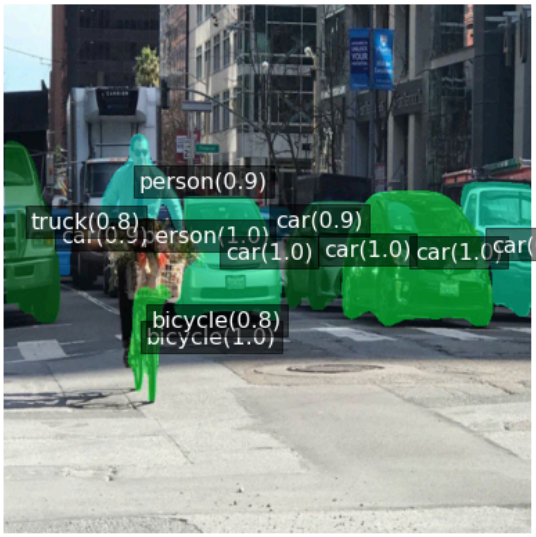



truck(0.8)
bicycle(0.8)
dog(1.0)

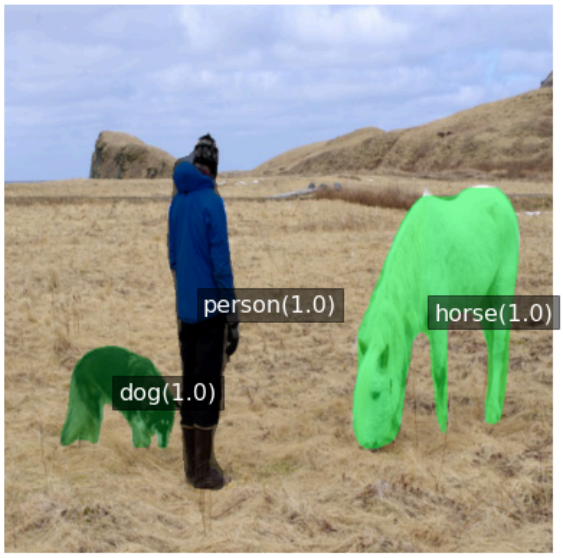



horse(0.8)
elephant(1.0)
elephant(1.0)

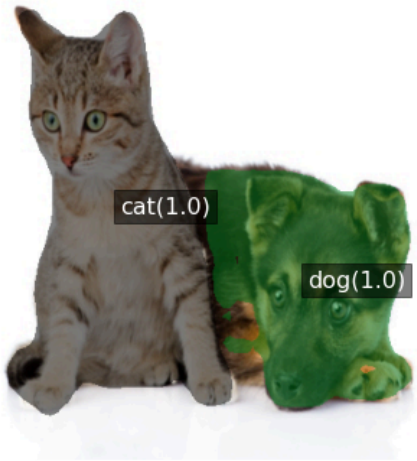


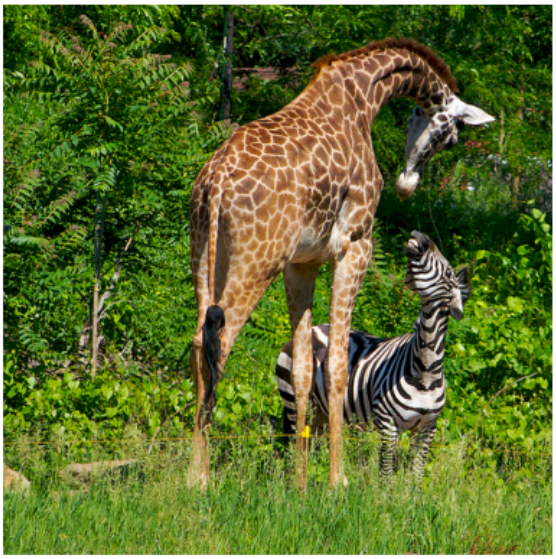

person(0.9)
truck(0.8)
car(0.9)
car(1.0)
car(1.0)
car(0.9)
bicycle(0.8)




person(1.0)
horse(1.0)
dog(1.0)

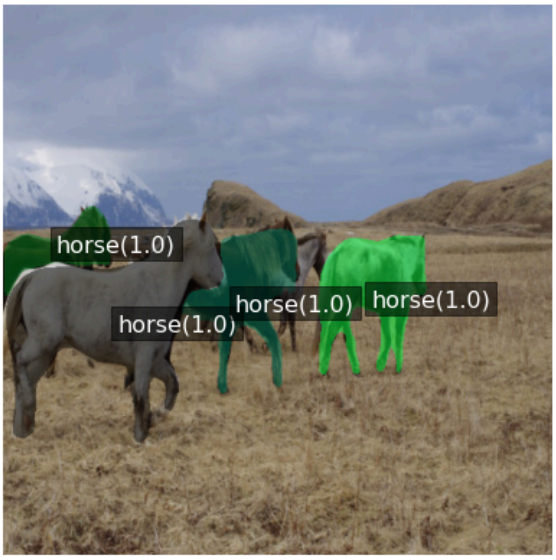



cat(1.0)
dog(1.0)




giraffe(1.0)
zebra(1.0)




horse(1.0)
horse(1.0)
horse(1.0)
horse(1.0)